

You are building a full-scale, industry-level enterprise AI platform called Enterprise AI Operating System for Teams. The existing project already has a strong MVP shape, but it must now be transformed from a mostly simulated enterprise AI workspace into a genuinely functional, production-minded platform with real backend intelligence, real workflows, real retrieval, real observability, real security controls, and a clean enterprise UX. The goal is not to make a simple chatbot or a demo dashboard. The goal is to build a serious, resume-defining, startup-grade AI operating system that could plausibly be used inside an organization as an internal AI work platform.

You must preserve the current overall product identity: a dark, dense, enterprise cockpit with a command center, knowledge layer, agents layer, workflows layer, memory layer, security layer, observability layer, and settings layer. However, the internals must be upgraded substantially. Anything that is currently only a mock, placeholder, or visual simulation must be replaced with real architecture wherever possible. Mock mode may remain as a safe fallback for demos, but the product must also have a real production path behind feature flags and provider adapters.

The implementation must prioritize depth over surface polish. Do not stop at making the UI look good. The important part is that the system behaves like a real enterprise AI platform: it should store data correctly, retrieve knowledge correctly, route work to agents correctly, protect sensitive operations, track every important action, and make the system observable and auditable.

Use the following technology direction unless a better choice is required for correctness or maintainability. The main app should remain a Next.js App Router + React + TypeScript platform with Tailwind CSS, shadcn/ui-style components, and lucide-react icons. The backend should continue to use the Next.js API/route-handler architecture where practical, but the backend logic must be organized cleanly into service layers, domain modules, and adapter layers rather than being embedded directly in UI code. Use Drizzle ORM with PostgreSQL as the persistent store, and use pgvector for vector retrieval. Add Redis for caching, background job coordination, task queues, rate limiting, and short-term state where appropriate. Use SSE for streaming responses and workflow events. If a background worker architecture is needed, introduce it in a clean way, preferably through a queue-based job system such as BullMQ or a similar Redis-backed worker model. If any specialized AI-processing service is needed later, design the interfaces so the app can connect to a separate service cleanly, but do not break the existing app structure.

The most important requirement is that the project must become a real enterprise AI system, not merely a mocked simulation.

Priority 1: Replace simulated multi-agent behavior with real orchestration

The current agent experience is not enough if it is only a scripted sequence of steps. Replace the existing simulated “agent trace” behavior with a real orchestration system that manages stateful, typed, auditable agent runs. Use an orchestration pattern such as LangGraph or an equivalent

graph/state-machine architecture. The system must support a planner, research agent, analyst agent, writer agent, security agent, and evaluator agent as real modular components with clear responsibilities, typed inputs, typed outputs, and persisted execution state. Agent runs must be durable, resumable when needed, traceable, and able to fail gracefully with retries or guarded aborts.

The planner must break down a task into structured subgoals. The research agent must retrieve evidence from the knowledge layer. The analyst must reason over the evidence and rank it. The writer must generate the final grounded response. The security agent must inspect prompts and execution plans for risky behavior, prompt injection attempts, unauthorized actions, or policy violations. The evaluator must score the answer for groundedness, citation coverage, retrieval quality, and risk. The orchestration must not simply render fake traces. It must persist real step states, real timestamps, real agent decisions, and real tool routing decisions. Each run must be inspectable in the UI and through the API.

The UI should still display an agent trace timeline, but the trace must now reflect actual orchestration state. Each step should clearly show status transitions such as queued, running, awaiting approval, succeeded, failed, skipped, or retried. The system should also support tool-using agents in a controlled and auditable way.

Priority 2: Build a real knowledge ingestion and retrieval system

The knowledge layer must become a genuine RAG pipeline, not a decorative search page. Document ingestion must support PDF, text, markdown, CSV, and JSON at minimum. The pipeline must validate file type, normalize and extract text, clean it, chunk it intelligently, generate embeddings, store document metadata and chunk records in PostgreSQL, and make the chunks searchable through vector retrieval. Use pgvector for production retrieval. If local hash embeddings remain available for demo mode, they must be clearly isolated as a mock provider, not the only implementation.

Retrieval must support semantic search plus metadata filtering. The system should allow the user to see the source documents, chunk snippets, similarity scores, and grounded citations that support the final answer. Add ranking logic, optional reranking, and retrieval evaluation. The goal is to reduce hallucination and make the system behave like a grounded enterprise knowledge assistant. The chat flow must retrieve evidence before generating an answer, and the response must include citations that map back to stored chunks and source documents.

The ingestion pipeline should be designed cleanly enough to support OCR later, but OCR should not be treated as a fake “done” feature if it is not actually implemented. Only claim functionality that is real. If OCR is not present, make that visible in the code structure as a future extension point rather than pretending it exists.

Priority 3: Add real tool integrations with a safe adapter architecture

The system must not rely only on fake tool execution. Build a proper tool adapter layer that can connect to external enterprise tools through stable interfaces. At minimum, support adapters for GitHub and one of Slack, Gmail, Jira, or Calendar in a real or clearly pluggable way. The architecture should allow these tools to be used in mock mode by default, but the production path should exist through environment-based configuration and scoped credentials.

Every tool call must be permissioned, auditable, and visible in a tool execution ledger. Some tool categories must require human approval before execution. Tool calls should support a lifecycle such as proposed, pending approval, approved, executed, failed, or cancelled. Tool adapters must not be hardcoded inside UI components. They should be implemented as service abstractions with request validation, output normalization, error handling, and audit logging.

The goal is for the platform to genuinely feel like an AI workforce that can take controlled actions, not only generate text. Even if the default runtime remains safe and sandboxed, the architecture must make real tool execution feasible and well-structured.

Priority 4: Implement a real workflow engine instead of a static workflow demo

The workflow layer must become a real asynchronous workflow engine with persisted state. The existing HR, finance, engineering, and operations workflows should be upgraded from simple scripted demos into real workflow definitions with step dependencies, retry policies, approval gates, and streamed status updates. Workflows should be modeled as durable runs that can be paused, resumed, retried, inspected, and audited.

The workflow engine should support task queues, scheduled jobs, worker processing, and event streaming to the UI. Each workflow step should have typed inputs, typed outputs, status transitions, execution logs, and error handling. The user should be able to launch a workflow, watch it progress, see where approvals are needed, approve or reject certain actions, and review the final audit trail.

Use a queue-based or event-driven architecture so the workflows are not just synchronous function calls. The implementation should be suitable for future horizontal scaling. If you use Redis queues or a worker model, persist the authoritative workflow run state in the database. The workflow engine must feel like a real automation system, not a slideshow of steps.

Priority 5: Build a genuine memory system with short-term, long-term, semantic, and episodic layers

The memory layer must evolve from simple conversation storage into a true enterprise memory system. At minimum, implement short-term conversation memory, long-term organizational memory, and a clear retrieval policy for both. Long-term memory should store relevant organizational preferences, recurring decisions, policy snippets, workflow outcomes, and durable context that can be recalled later. The system should also support semantic memory indexed by embeddings so that prior decisions and important knowledge can be rediscovered through meaning rather than exact text match.

Memory should be scoped to the organization and should never cross tenant boundaries. It must support updates, retrieval, and summarization policies so the system does not accumulate useless noise. Add an abstraction that lets the system write memory when a conversation, workflow, or tool action produces a durable insight. Also add policies for memory decay, importance scoring, and summarization so memory remains useful over time.

The UI should show meaningful memory records and not just a raw log dump. The memory area should help explain what the system remembers, why it remembers it, and how that memory influences future answers or workflows.

Priority 6: Upgrade observability and evaluation into a real engineering surface

The observability layer must be treated as a first-class product feature. Add structured logs, request traces, workflow traces, agent traces, token usage tracking, latency metrics, retrieval quality metrics, hallucination risk scoring, and approval-gate metrics. The platform should record not only what answer was generated, but also how it was generated, what evidence was used, whether the result was grounded, whether the retrieval was good enough, and whether any risky action was blocked or approved.

Add a proper AI evaluation pipeline. It must at least track groundedness, citation coverage, retrieval quality, response latency, model/provider usage, and any policy or security flags encountered during execution. Build the observability UI so that it is not just decorative. It should help an engineer inspect what happened, where the system is weak, and whether the answer can be trusted.

If possible, add integration-friendly hooks for Prometheus-style metrics, OpenTelemetry-style traces, or a similar monitoring system. The app should be able to expose health checks and readiness checks. The observability pages should look and behave like internal engineering tooling used by a serious team.

Priority 7: Strengthen security, governance, and tenant isolation

Security must be treated as a core product requirement, not an afterthought. Maintain and improve signed HTTP-only sessions, role-based access control, protected routes, rate limiting, and tenant-scoped records. Every core entity should be organization-scoped. No tenant data must leak into another tenant's view, API, memory, workflow, or audit surface. Add input validation everywhere, especially on uploads, prompt inputs, tool calls, and workflow operations.

Implement a clear security layer for prompt injection detection, suspicious instruction detection, sensitive data masking, approval-gate enforcement, and unsafe action prevention. When an incoming prompt or document appears suspicious, the system should flag it and route it through the security layer instead of blindly processing it. Tool execution should be blocked or gated when risk is elevated.

Add secure-by-default operational behavior. Use environment variables for secrets, do not commit sensitive credentials, and ensure the app can run locally in mock-safe mode without exposing production capabilities. Provide clear security defaults in the settings page, including what is enabled, what is disabled, and what requires approval.

Priority 8: Make the app production-ready in engineering terms

The project must not only function; it must be engineered responsibly. Add robust type safety, schema validation, error handling, status handling, and build discipline. Keep the codebase modular and maintainable. Separate UI components, domain logic, adapters, services, and persistence layers. Use reusable types and validation schemas for all major inputs and outputs. Maintain consistency in naming, folder structure, and API contracts.

Add tests that cover critical behavior. At minimum, include tests for authentication and RBAC, document ingestion and retrieval, agent orchestration, workflow execution, tool approval gating, and security filtering. Add integration tests for main API routes. Ensure the project passes lint, typecheck, and production build checks.

Add a clean migration strategy for the database schema. Include seed data for demo users, seeded documents, and sample workflows so the app can be demonstrated without manual setup. Add clear environment setup instructions and provide a strong local developer experience using Docker Compose for PostgreSQL and any necessary services.

Priority 9: Improve the UX so it feels like an internal enterprise product

Keep the dense enterprise cockpit style, but make the experience more polished and more informative. The app should feel like an internal AI control room, not a marketing website. Preserve

the command center dashboard, side navigation, top status bar, and dark professional theme. Improve loading states, empty states, error states, streaming states, approval states, and success states so the product feels alive and trustworthy.

The main dashboard should clearly show operational metrics such as documents indexed, workflow runs, agent success rate, approvals pending, memory records, retrieval quality, and cost estimates. The chat area should clearly indicate what agent flow is running, what context was retrieved, what citations were used, whether the answer is grounded, and whether security issues were detected. The knowledge page should make ingestion and search feel real. The agents page should make orchestration explainable. The workflows page should make execution and approvals easy to inspect. The memory, security, observability, and settings pages should each provide real operational insight, not only decorative cards.

Responsive behavior must be good, accessibility must be respected, and the interface should remain useful on smaller screens without losing the enterprise feel.

Priority 10: Turn the project into a complete portfolio-grade system with documentation and demos

The final result must be something you can confidently present in interviews. Add a strong README, architecture documentation, a clear setup flow, and a narrative explaining what the system does and why it is technically meaningful. Include a realistic demo scenario that walks through uploading a document, asking a question, seeing retrieval and citations, launching a workflow, approving a tool call, and inspecting observability data.

Add example seeded documents and workflows that show the platform doing meaningful work. Add clear explanations of the agent architecture, RAG architecture, workflow engine, memory model, security model, and observability model. Make sure the repo and the product are understandable to a recruiter, interviewer, or engineering manager who is evaluating whether this is a serious systems project.

Non-negotiable implementation rules

Do not remove existing working functionality unless it is being replaced by a better real implementation. Do not pretend that a mocked feature is production-ready. When mock mode is used, label it honestly and keep it separate from the real path. Do not hardcode secrets. Do not allow one tenant to access another tenant's data. Do not let the UI become bloated with unnecessary marketing content. Do not leave core routes empty or fake. Do not stop at visuals. The implementation must be correct, maintainable, and deeply structured.

Definition of done

The project is done only when the following are true. The app has a real authentication and RBAC flow. The knowledge system actually ingests, stores, retrieves, and cites documents. The agent system actually orchestrates real step-based runs. The workflow engine actually creates and tracks workflow executions. Tool actions are routed through a proper approval and audit path. Memory is stored and retrieved in a meaningful way. Observability captures real execution data. Security and tenant isolation are enforced across the system. The app builds cleanly, has sensible tests, and runs as a coherent enterprise product. The overall experience should be good enough to present as a high-level engineering project for a serious resume

You are now acting as a principal-level AI systems engineer, staff backend engineer, distributed systems architect, AI infrastructure engineer, and enterprise SaaS product engineer simultaneously.

You are responsible for transforming the existing MVP of Enterprise AI Operating System for Teams into a truly functional, production-minded, industry-grade enterprise AI platform.

This project must evolve from a partially simulated AI workspace into a real multi-agent enterprise operating system with actual orchestration, retrieval, memory, workflow execution, observability, security enforcement, and controlled tool execution.

The project must remain developer-friendly, modular, scalable, auditable, explainable, secure-by-default, and architecturally mature.

The end result should look and behave like:

an internal enterprise AI platform,
an AI operations cockpit,
an AI workforce management system,
or an early-stage startup SaaS platform.

The project must be strong enough to:

dominate technical interviews,
stand out on elite engineering resumes,
survive deep architectural questioning,

and demonstrate genuine systems engineering capability.

The implementation must prioritize:

correctness,
architecture,
maintainability,
security,
observability,
scalability,
engineering depth,
before cosmetic polish.

CORE PRODUCT IDENTITY

The platform is:

“Enterprise Brain + AI Workforce”

The system must provide:

AI-assisted work execution,
enterprise knowledge retrieval,
multi-agent orchestration,
workflow automation,
memory persistence,
AI observability,
secure tool execution,
organizational intelligence,
and enterprise governance.

This is NOT:

a chatbot wrapper,
a simple RAG app,
a fake dashboard,
or a UI-only simulation.

This IS:

a distributed AI workspace platform.

NON-NEGOTIABLE ENGINEERING PRINCIPLES

RULE 1 — NEVER FAKE CORE FUNCTIONALITY

Do not simulate functionality that should actually exist.

If a feature is mocked:

clearly isolate it,
clearly label it,
and implement a real production path.

Mock mode is allowed ONLY as:

a safe demo mode,
a fallback development mode,
or an offline mode.

The production path must always exist architecturally.

RULE 2 — SEPARATE CONCERNS CLEANLY

The codebase must NOT become:

monolithic,
UI-driven,
tightly coupled,
or difficult to maintain.

Strictly separate:

UI,
orchestration,
retrieval,
domain logic,
memory,
workflows,
providers,
tools,
persistence,
observability,
and security.

RULE 3 — EVERYTHING IMPORTANT MUST BE AUDITABLE

Every major system action must be traceable:

agent runs,
workflow execution,
tool execution,
memory writes,
security flags,
approvals,
retrieval decisions,
provider selection,

failures,
retries.

The system should behave like enterprise infrastructure.

TARGET ARCHITECTURE

The final architecture must approximately follow:

Frontend Layer



API Gateway Layer



Orchestration Layer



Agent Runtime Layer



Retrieval + Memory Layer



Workflow Execution Layer



Tool Adapter Layer



Persistence + Observability Layer

All major systems must be modular.

MANDATORY TECHNOLOGY STACK

Frontend

Next.js App Router

React

TypeScript

Tailwind CSS

shadcn/ui

Lucide-react

Zustand or Redux Toolkit for complex client state

Framer Motion where appropriate

SSE for streaming

WebSockets only if truly necessary

Backend

Next.js route handlers initially

Service-oriented architecture internally

Typed domain modules

Clean adapter pattern

Zod validation everywhere

Typed API contracts

Database

PostgreSQL

Drizzle ORM

pgvector extension

Queue + Background Processing

Redis

BullMQ preferred

AI Infrastructure

OpenAI

Anthropic

Ollama support

Hugging Face support

Provider abstraction layer mandatory

Retrieval

pgvector initially

optional Qdrant adapter later

Observability

Structured logs

OpenTelemetry-ready traces

Metrics abstraction

Evaluation pipelines

Deployment

Docker Compose

Vercel-compatible frontend

containerized backend services

environment-driven configuration

PHASE 1 — FOUNDATION HARDENING

OBJECTIVE

Stabilize architecture before adding advanced intelligence.

REQUIRED TASKS

1. Refactor the entire codebase into clean domains

Create clear modules:

/modules/auth

/modules/agents

/modules/workflows

/modules/retrieval

/modules/memory

/modules/security

/modules/tools

/modules/observability

/modules/evaluation

/modules/providers

/modules/documents

Each module must contain:

services,
repositories,
schemas,
types,
validators,
adapters,
APIs,
utilities.

Do NOT place business logic inside React components.

2. Add strict validation everywhere

Use:

Zod schemas,
typed DTOs,
request validation,
response validation.

Validate:

uploads,
prompts,
workflow inputs,
tool execution,
provider configs,
memory writes.

Reject malformed input safely.

3. Add robust error architecture

Implement:

typed errors,
retryable errors,
security errors,
provider errors,
workflow errors,
validation errors.

Add centralized error handling.

No silent failures.

4. Add production-grade logging

Implement structured logging for:

requests,
workflows,
agents,
retrieval,
tools,
security,
memory.

Every log must include:

timestamp,
organizationId,
requestId,
userId,
severity,
component.

5. Add proper environment management

Implement:

typed env validation,
secrets separation,
provider toggles,
feature flags,
runtime configuration.

ACCEPTANCE CRITERIA — PHASE 1

The app:

builds cleanly,
passes lint,
passes typecheck,
has modular structure,
has centralized logging,
has validation everywhere,
and has no major architectural chaos.

PHASE 2 — REAL MULTI-AGENT ORCHESTRATION

OBJECTIVE

Replace simulated traces with real orchestration.

REQUIRED ARCHITECTURE

Implement:

graph-based orchestration,
durable execution state,
step-based execution,
resumable runs,
retryable execution.

Preferred:

LangGraph.

Alternative:

custom state machine orchestration.

REQUIRED AGENTS

Planner Agent

Responsible for:

decomposing goals,
planning execution,
assigning subtasks.

Research Agent

Responsible for:

retrieval,

search,
evidence gathering.
Analyst Agent

Responsible for:

evidence ranking,
reasoning,
anomaly detection,
structured analysis.
Writer Agent

Responsible for:

grounded synthesis,
answer generation,
report formatting.
Security Agent

Responsible for:

prompt injection detection,
unsafe execution blocking,
policy analysis.
Evaluator Agent

Responsible for:

groundedness scoring,
hallucination detection,
retrieval quality scoring.

EXECUTION MODEL

Every run must persist:

- status,
- steps,
- timestamps,
- provider used,
- latency,
- token usage,
- tool calls,
- memory writes,
- citations,
- failures.

The UI trace must reflect REAL execution state.

ADDITIONAL REQUIREMENTS

Implement:

- retries,
- cancellation,
- timeout handling,
- partial failures,
- execution recovery.

ACCEPTANCE CRITERIA — PHASE 2

The system:

- genuinely orchestrates agents,

persists runs,
streams live execution,
and produces real traces.

No fake traces allowed.

PHASE 3 — REAL RAG + KNOWLEDGE SYSTEM

OBJECTIVE

Build a real enterprise retrieval layer.

INGESTION PIPELINE

Implement:

upload validation,
text extraction,
normalization,
semantic chunking,
embedding generation,
metadata indexing,
vector storage.

Supported:

PDF

TXT

MD

CSV

JSON

RETRIEVAL PIPELINE

Implement:

semantic retrieval,
metadata filtering,
chunk ranking,
optional reranking,
citation mapping,
retrieval scoring.

Responses must show:

evidence,
sources,
chunk references,
retrieval confidence.

EMBEDDING PROVIDERS

Support:

OpenAI embeddings,
SentenceTransformers,
local embeddings.

Must support provider abstraction.

DATABASE REQUIREMENTS

Store:

documents,

chunks,
embeddings,
metadata,
citations,
retrieval logs.

ACCEPTANCE CRITERIA — PHASE 3

The platform:

retrieves real context,
grounds answers,
shows citations,
and reduces hallucinations.

PHASE 4 — REAL WORKFLOW ENGINE

OBJECTIVE

Build a true workflow automation engine.

REQUIRED FEATURES

Implement:

persisted workflow definitions,
workflow runs,
step execution,
async processing,
retries,
approvals,
scheduling,
event streaming.

EXECUTION ARCHITECTURE

Use:

BullMQ,
Redis queues,
worker processors.

Persist authoritative state in PostgreSQL.

WORKFLOW TYPES

Support:

HR workflows,
finance workflows,
engineering workflows,
operations workflows.

APPROVAL SYSTEM

Implement:

approval-required steps,
approvers,
rejection handling,
execution pausing.

ACCEPTANCE CRITERIA — PHASE 4

The workflows:

execute asynchronously,
survive failures,

stream progress,
and maintain audit history.

PHASE 5 — REAL TOOL EXECUTION LAYER

OBJECTIVE

Build controlled enterprise integrations.

REQUIRED TOOL ADAPTERS

Minimum:

GitHub

Slack OR Gmail

Calendar

TOOL EXECUTION MODEL

Every tool action must support:

proposed,
pending approval,
approved,
executed,
failed,
cancelled.

SECURITY REQUIREMENTS

Implement:

scoped permissions,
approval gates,
audit trails,

execution validation.

TOOL ARCHITECTURE

Use adapter pattern:

no hardcoded providers,

no provider logic inside UI.

ACCEPTANCE CRITERIA — PHASE 5

The system:

can execute real actions,

safely,

auditably,

and controllably.

PHASE 6 — MEMORY SYSTEM

OBJECTIVE

Implement real organizational memory.

REQUIRED MEMORY TYPES

Short-Term Memory

Conversation/session memory.

Long-Term Memory

Persistent organizational memory.

Semantic Memory

Embedding-searchable knowledge.

Episodic Memory

Workflow/event history.

REQUIRED FEATURES

Implement:

memory importance scoring,

memory retrieval,

semantic recall,

summarization,

memory aging,

organization scoping.

ACCEPTANCE CRITERIA — PHASE 6

The system:

meaningfully remembers,

retrieves context intelligently,

and improves continuity.

PHASE 7 — OBSERVABILITY + AI EVALUATION

OBJECTIVE

Build true AI operations visibility.

REQUIRED METRICS

Track:

token usage,
latency,
retrieval quality,
groundedness,
hallucination risk,
approval frequency,
workflow success,
provider usage.

REQUIRED FEATURES

Implement:

traces,
dashboards,
structured metrics,
evaluation records,
runtime readiness,
health checks.

EVALUATION PIPELINE

Score:

groundedness,
citation coverage,
answer relevance,
retrieval relevance,
hallucination probability.

ACCEPTANCE CRITERIA — PHASE 7

The system:

exposes operational insight,
and helps engineers debug AI behavior.

PHASE 8 — SECURITY + GOVERNANCE

OBJECTIVE

Make the platform enterprise-safe.

REQUIRED FEATURES

Implement:

RBAC,
tenant isolation,
signed sessions,
rate limiting,
prompt injection detection,
sensitive data masking,
audit logging,
secure secrets handling.

SECURITY ENFORCEMENT

Every:

workflow,
tool call,
memory access,
retrieval,
provider request,
must respect organization boundaries.

ACCEPTANCE CRITERIA — PHASE 8

The system:

behaves securely,
isolates tenants,
and blocks unsafe behavior.

PHASE 9 — DEPLOYMENT + DEVOPS

OBJECTIVE

Make the system production-deployable.

REQUIRED FEATURES

Implement:

Docker Compose,
production configs,
CI/CD,
migration pipelines,
seeded demo environments,
health endpoints,
readiness checks.

TESTING

Add:

unit tests,
integration tests,
API tests,
workflow tests,
orchestration tests.

ACCEPTANCE CRITERIA — PHASE 9

The app:

deploys cleanly,
survives restarts,
and runs reliably.

PHASE 10 — PORTFOLIO POLISH

OBJECTIVE

Turn the system into an elite engineering portfolio project.

REQUIRED DELIVERABLES

Create:

architecture diagrams,
execution diagrams,
workflow demos,
README,
API documentation,
setup guide,
deployment guide,
engineering rationale.

DEMO FLOW

The demo should show:

document upload,
ingestion,
retrieval,

grounded chat,
workflow launch,
approval gate,
tool execution,
observability trace,
memory recall.

FINAL QUALITY BAR

The finished platform must feel like:

a startup MVP,
internal enterprise tooling,
or a serious AI infrastructure platform.

It must NOT feel like:

a tutorial,
a dashboard mockup,
a chatbot wrapper,
or a fake AI demo.

Every major system must:

actually work,
persist state,
be observable,
and be explainable.

The codebase must demonstrate:

systems engineering,

backend architecture,
AI infrastructure,
production thinking,
security awareness,
and enterprise software maturity.

This project is intended to become:

a flagship resume project,
a deep technical discussion project,
and a proof of high-level engineering capability.

Do not cut corners.